



Rifugio per adozione animali

<http://localhost/>

Relazione di progetto

Università degli Studi di Padova
L.T. in Informatica
A.A. 2025/26

2111001 (Referente) Thomas Morettin (thomas.morettin@studenti.unipd.it)
2111935 Felician Mario Necsulescu (felicianmario.necsulescu@studenti.unipd.it)
2111947 Niccolò Feltrin (niccolo.feltrin@studenti.unipd.it)

Data 02.02.2026
Corso Tecnologie Web
Cod. SCP4065581

Credenziali di accesso al sito

Ruolo	Username	Password
Amministratore	admin	admin

Indice

1	Scopo del sito (Felix)	2
2	Analisi dei requisiti (Felix)	2
3	Progettazione (Felix)	2
4	Web Development	2
4.1	HTML5	2
4.2	CSS3	2
4.3	JavaScript	3
4.4	PHP	3
4.4.1	Pattern MVC	3
4.4.2	utils.php	3
4.4.3	Composer (vendor)	4
4.4.4	db-access.php	4
4.4.5	template.php, header.php e breadcrumb.php	4
4.4.6	.htaccess	4
4.5	Base di dati	5
4.6	Sicurezza	6
5	Accessibilità e usabilità	6
5.1	Elementi fondamentali	6
5.2	Orientamento e navigazione	6
5.3	WAI-ARIA	6
5.4	Colori e Brand Identity	6
6	Test e validazione (Niccolò)	6
7	SEO e prestazioni	6
7.1	faq-schema.js	6
7.2	org-schema.js	6
7.3	Web Performance Optimization	6
8	Organizzazione (Niccolò)	6
9	Difficoltà (Niccolò)	6
10	Conclusioni (Niccolò)	6

1 Scopo del sito (Felix)

2 Analisi dei requisiti (Felix)

3 Progettazione (Felix)

4 Web Development

All'interno di questa sezione si vogliono descrivere le tecnologie, gli standard e la struttura adottati all'interno del progetto.

4.1 HTML5

Le pagine del sito web sono state create facendo uso di HTML5 (HyperText Markup Language 5), ovvero lo standard attuale per la progettazione di siti web. In particolare sono state adottate le seguenti best practices di programmazione:

- SoC (Separation of Concerns), in quanto si è mantenuta la separazione tra la struttura (HTML5), lo stile (CSS) e la logica di funzionamento (JavaScript/PHP), rendendo il prodotto scalabile e manutenibile;
- metatag, ovvero snippet per definire metadati utili al motore di ricerca e ai browser, incrementando la SEO (Search Engine Optimization) e rendendo il sito accessibile;
 - è stato fatto uso, oltre a quelli comunemente utilizzati, del metatag `format-detection` per disabilitare l'abilitazione di un link a numero di telefono, da parte del browser, su un numero che non lo sarebbe (pena disorientamento dell'utente).
- separazione degli elementi, in quanto si è deciso di adottare un approccio scalabile, rendendo elementi come i dialog, l'header, il footer e la breadcrumb unici all'interno di tutto il sito, potendoli riutilizzare in più pagine differenti mantenendone una gestione, sia semantica che di comportamento, centralizzata;
- validazione delle pagine al sito W3C Validator per mantenere una semantica corretta e garantire l'accessibilità.

Con questo linguaggio di markup è stato possibile strutturare il corpo dell'e-mail automatica che viene inviata dal server all'utente che compila una richiesta tramite form all'interno del sito. Allo stato attuale i server di posta non supportano una struttura moderna, ma bensì obsoleta, richiedendo l'uso di tabelle per la strutturazione del contenuto o, peggio, di attributi non coerenti con gli standard correnti. Il tutto, però, è stato gestito con la massima attenzione all'accessibilità e alla struttura, rendendo comunque l'e-mail fruibile e leggibile da chiunque.

4.2 CSS3

Per la presentazione del sito web è stato fatto uso del linguaggio CSS3 (Cascading Style Sheets 3), con la costruzione di layout sia flexbox che grid per mantenere una struttura scalabile e pulita all'interno delle pagine. In particolare, il foglio di stile principale `style.css` viene utilizzato per definire un layout generale desktop, permettendo poi al sito di degradare in modo fluido quando viene visualizzato su schermo inferiore ai 768px di larghezza grazie alle ridefinizioni nel foglio `style-smartphone.css`. Il linguaggio è stato utilizzato, inoltre, per gestire la stampa delle pagine e garantire che vengano impaginate correttamente.

A livello presentazione è stata utilizzata l'unità fissa px solamente per ombre, bordi di elementi e larghezza massima della pagina (1200px); per i restanti elementi (l'estrema maggioranza) solamente unità dinamiche come rem, em, %, e vh.

Con questo linguaggio è stato possibile definire la presentazione del corpo dell'e-mail automatica precedentemente citata. Gli standard in uso per questa tecnologia, come già detto risultano obsoleti e richiedono l'utilizzo di stili inline per poter essere applicati correttamente e adattarsi ai principali servizi di posta. Nonostante ciò è stato possibile mantenere la separazione tra la struttura e la presentazione (`style-email.css`).

4.3 JavaScript

Per la gestione del comportamento delle pagine si è fatto uso di JavaScript. In particolare si è optato per includere all'interno di ciascuna pagina web un file `global.js`, il quale contiene le seguenti funzioni più importanti:

- gestione della funzionalità di rimozione/reinserimento dello sfondo delle pagine (gestita tramite variabile d'ambiente);
- gestione della funzionalità tema chiaro/scuro (gestita tramite variabile d'ambiente);
- gestione della comparsa e definizione grafica dell'elemento toast, utilizzato per l'annuncio all'utente di operazioni andate o meno a buon fine;
- funzione di "caricamento" degli elementi grafici fondamentali, in quanto se il JavaScript dovesse essere disabilitato mostrerebbe elementi all'utente senza un funzionamento (es. con JS disabilitato è impossibile ritornare in cima alla pagina, il pulsante viene nascosto);
- funzione di cambio anno di copyright dinamico.

Successivamente, ciascuna pagina possiede un file JavaScript proprio per la dichiarazione di event listeners, funzioni di controllo degli input e della gestione degli errori. Questa scelta di separazione tra un file globale e quelli specifici è stata adottata per rendere le pagine meno pesanti a livello di memoria, dichiarando elementi solo in caso di necessità. Ulteriori file aggiuntivi possono essere caricati tramite placeholder `{{extra-js}}`, come per esempio il codice che gestisce il funzionamento del pannello di filtri o di script `org` per il potenziamento della SEO.

4.4 PHP

A livello server-side è stato fatto uso di linguaggio PHP (Hypertext PreProcessor). Gli script rappresentano il motore di tutte le pagine del sito, in quanto la creazione della pagina è centralizzata all'interno del file `utils.php`, il quale permette di generare, a partire da un template base, le pagine a struttura standardizzata. Successivamente, le interazioni con la base di dati e la conseguente creazione degli elementi ad essa dipendenti vengono gestite da file PHP dedicati per ciascuna pagina.

4.4.1 Pattern MVC

All'interno dell'ambito programmazione il Model View Controller è il design pattern più rinomato ed efficiente in quanto di gestione ed organizzazione dei compiti per ciascuno script interno al progetto. In particolare si riconoscono le seguenti categorie:

- Model: interfaccia che permette l'interazione con la base di dati; all'interno del progetto afferiscono a questa categoria tutti i file PHP che, dopo aver instaurato la connessione con il DB, hanno il compito di estrapolare i dati tramite query parametrizzate, sanitizzare l'output e restituire allo script chiamante i risultati;
- View: insieme dei file HTML che presentano lo scheletro per la strutturazione dei dati da visualizzare;
- Controller: insieme di script PHP adibiti alla manipolazione dei risultati restituiti dal Model; questi script hanno il compito di generare markup strutturale coerente con i dati ricevuti ed arricchire così lo scheletro HTML con dati strutturati e organizzati; oltre ai file per la creazione di elementi dinamici nella pagina HTML, afferiscono a questa categoria anche i file utilizzati per l'invio dei dati alla base di dati, con controllo dell'input a livello server-side (massimizzando il controllo già previsto lato client tramite JS).

4.4.2 `utils.php`

Progettato per essere il file "cuore" del motore che permette la costruzione delle pagine esso presenta le seguenti funzionalità principali:

- dichiara univocamente all'interno di tutto il progetto la root del sito web, permettendone la sostituzione al placeholder `{{root}}` o potendo essere riutilizzata all'interno di altri script PHP;

- funzione per il controllo di login da parte dell'utente tramite parametro di sessione e corrispondenza della chiave criptata per la password;
- funzione per la ricezione del messaggio di sessione (success/error) per il popolamento dell'elemento toast; il testo viene caricato all'interno della variabile `data-page` del tag `body`, utilizzabile anche per generici valori di sessione;
- funzione per la costruzione della pagina a partire da un template strutturato all'interno del file `template.html`; nel template vengono caricati: header, breadcrumb e footer (definiti in file `.html` appositi); all'interno di questa funzione si evidenziano i seguenti comportamenti:
 - nel caso in cui sia presente il placeholder `{{extra-js}}` vengono creati i tag di `<script>` appositi per incorporare più file JS contemporaneamente;
 - sostituzione di tutti i placeholder della pagina con eliminazione finale di quelli non sostituiti;
- funzione di parsing del file `request-mail.html`, contenente la struttura dell'email automatica, per "iniettare" i parametri di stile all'interno dei tag HTML tramite libreria Emogrifier; come spiegato in precedenza risulta essere il modo più efficiente per mantenere la compatibilità tra i diversi server di posta;
- funzione per l'invio automatico dell'email alla compilazione dei form di adozione o porta in adozione tramite libreria PHPMailer; l'invio avviene mediante protocollo SMTP, server Gmail (dominio di posta dell'email), con password per applicazione e completamento del destinatario.

4.4.3 Composer (vendor)

È stato fatto uso di Composer come programma applicativo per l'installazione delle librerie necessarie all'interno del progetto di Emogrifier (CSS inline parser) e PHPMailer (composizione automatica dell'email). Questa scelta permette la scalabilità del progetto, in quanto risulta semplice l'integrazione di nuove librerie senza l'obbligo di scaricarle manualmente, e la delegazione del compito di gestione delle dipendenze tra, per l'appunto, librerie.

4.4.4 db-access.php

File PHP per la gestione dell'apertura e chiusura della connessione alla base di dati. Lo script implementa le funzioni di apertura, chiusura e restituzione della connessione, insieme alla funzione di esecuzione della query che ha il compito di gestirne i parametri in ingresso, preparare lo statement e renderlo innocuo a SQL Injection e restituire uno/più risultati in base alla tipologia (SELECT, UPDATE, DELETE) della query.

4.4.5 template.php, header.php e breadcrumb.php

Come descritto precedentemente ciascun elemento della pagina viene gestito distintamente. Il file `template.php` permette di creare lo scheletro contenente tag `<head>` popolato e `<body>` da popolare.

Il file `header.php` presenta le funzioni per popolare i link di navigazione, con loro marcatura come link o meno, per inserire il link alla home nel logo e generare i pulsanti di amministrazione nel caso l'utente si sia autenticato.

Il file `breadcrumb.php` contiene al suo interno la definizioni dell'albero dei percorsi e come devono essere formattati all'interno dell'elemento breadcrumb, fondamentale per evitare il disorientamento dell'utente.

4.4.6 .htaccess

Il seguente file permette le operazioni di rooting del sito web. Nello specifico permette:

- visualizzazione delle pagine di errore personalizzate al posto della schermata predefinita del server per gli errori 500 e 404;
- pulizia dell'URL di pagina con rimozione dell'estensione `.php`;
- definizione dell'albero delle pagine, come descritto dall'elemento breadcrumb, senza possibilità di visualizzare i percorsi interni al repository di progetto.

4.5 Base di dati

Per il corretto funzionamento del sito web è stata implementata una base di dati in linguaggio MySQL all'interno del RDBMS Maria DB, la quale presenta al suo interno tutte le entità fondamentali per gestire le richieste degli utenti per l'adozione degli animali e la possibilità di portarli in adozione al rifugio; il tutto può essere centralizzato per l'amministratore tramite gestione delle richieste, una volta programmate, in calendario.

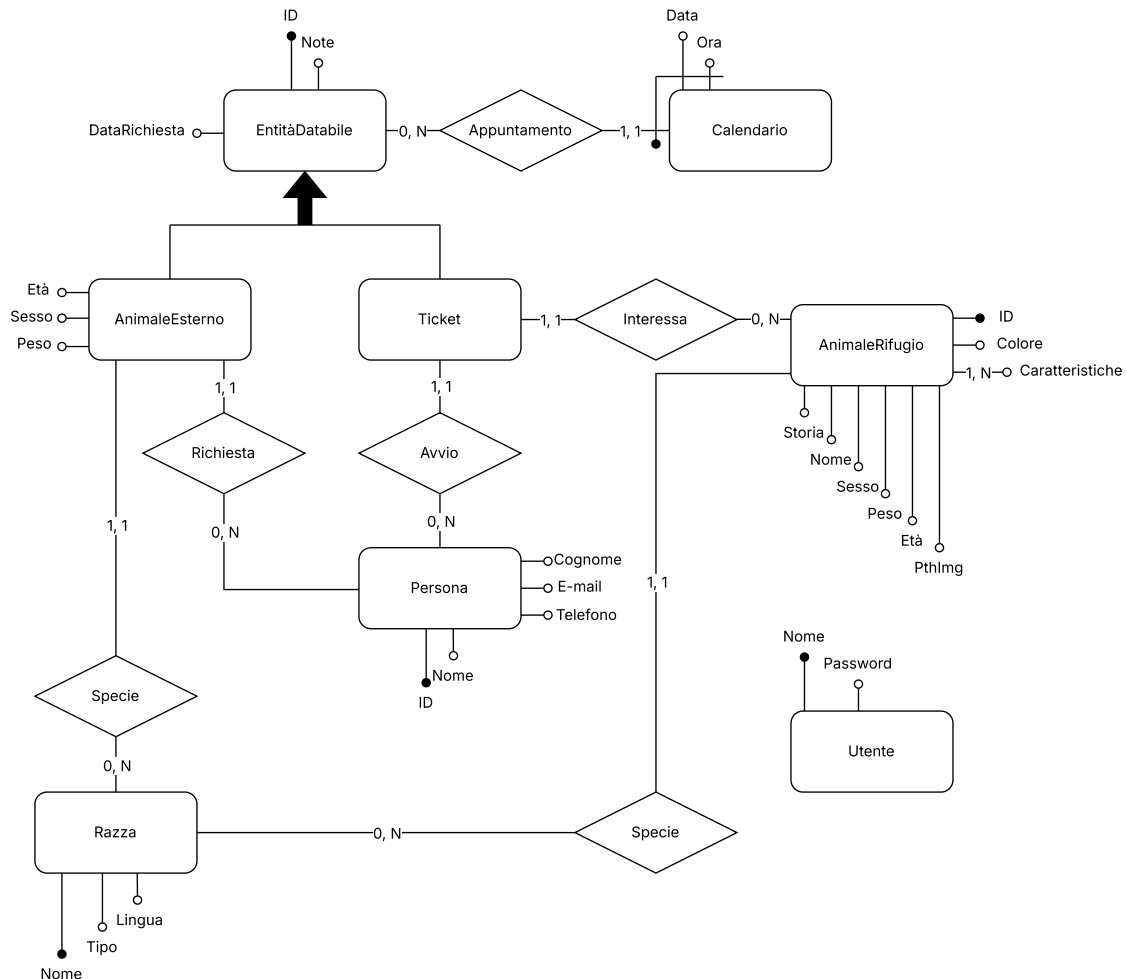


Figura 1: Diagramma ER della base di dati

Per questioni di spazio, ma volendo comunque approfondire il diagramma soprariportato, si approfondiscono le seguenti scelte progettuali:

- l'entità **EntitàDatabile** viene utilizzata per accorpare gli attributi comuni di **AnimaleEsterno** e **Ticket**, ovvero la richiesta di adozione; in particolare l'entità serve per permettere di calendarizzare le tuple, delle tabelle distinte, all'interno dell'entità **Calendario**, potendo usare ID univoco;
- l'entità **Utente** presenta un campo Password criptato...;
- l'entità **Razza** presenta un campo Lingua che viene utilizzato per popolare l'attributo lang HTML e poter quindi rendere accessibile la pronuncia di una razza nel momento in cui viene utilizzata all'interno del sito;
- l'entità **AnimaleRifugio** presenta il campo Età di tipo FLOAT(2, 2), permettendo così di estrapolare dalle cifre unitarie gli anni dell'animali, mentre da quelle decimali i mesi di quest'ultimo; questa scelta permette di filtrare anche per mesi gli animali del Rifugio.

4.6 Sicurezza

Per quanto concerne la sicurezza del sito web sono state applicate le seguenti tecniche di prevenzione danni nei confronti delle tecnologie client/server-side:

- creazione, come già parlato precedentemente, della funzione `exeQuery` la quale fa uso di `mysqli_prepare()` per parametrizzare le query e trattare l'input, eccetto dati integer/double, come una semplice stringa; ciò previene SQL Injection;
- password dell'amministratore salvata con tecnica di criptazione...;
- protezione da attacchi di tipo XSS (Cross-Site Scripting) grazie alla funzione `htmlspecialchars()` inserita all'interno di ogni funzione per l'estrapolazione dei dati dal database ed evitare l'inserimento di elementi HTML che potrebbero rompere la struttura della pagina.

5 Accessibilità e usabilità

5.1 Elementi fondamentali

5.2 Orientamento e navigazione

5.3 WAI-ARIA

5.4 Colori e Brand Identity

6 Test e validazione (Niccolò)

7 SEO e prestazioni

7.1 `faq-schema.js`

7.2 `org-schema.js`

7.3 Web Performance Optimization

8 Organizzazione (Niccolò)

9 Difficoltà (Niccolò)

10 Conclusioni (Niccolò)